

Isaiah Adams

Computer Science B.S. | Software Developer | Application Analyst

Professional Summary

Computer Science graduate with experience designing and developing software applications, managing technical projects, and building technology solutions using modern development practices. Experienced in full-stack development, database design, API integration, and collaborative software development through academic and team-based projects.

About This Portfolio

This portfolio provides additional details about software projects I have developed throughout my Computer Science studies. Each project highlights the problem addressed, solution implemented, technologies used, my role, key features, and technical challenges encountered during development.

Contact & Links

Email: IJAdams1@outlook.com

GitHub: github.com/iadams1

LinkedIn: [Linkedin.com/in/isaiah-j-adams/](https://www.linkedin.com/in/isaiah-j-adams/)

MONEYUP – CAPSTONE PROJECT

🔗 Project Links

Application Demo

View the MoneyUP application walkthrough

<https://youtu.be/7nN0hjX7TAc>

Source Code

Explore the GitHub repository

<https://github.com/iadams1/MoneyUP>

Demo QR Code



📄 Problem

Many college students struggle with budgeting, expense tracking, debt management, and developing financial literacy skills. Existing financial applications often target general audiences and include complex features that can overwhelm students who are learning to manage finances independently.

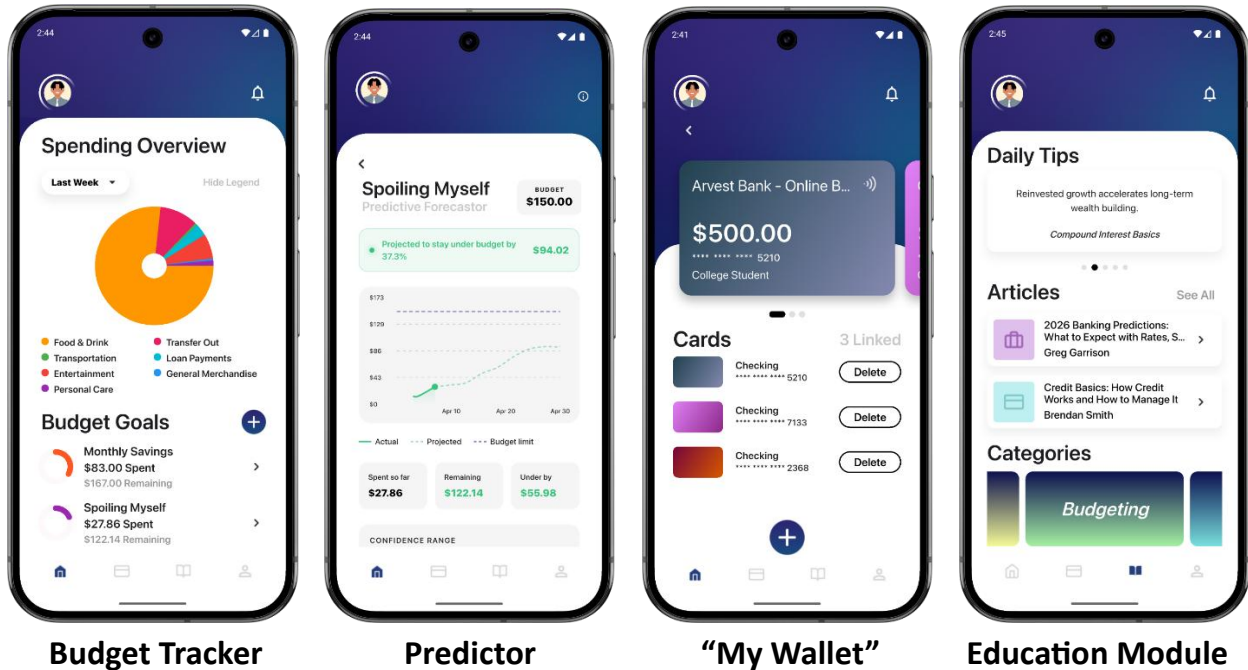
✓ Solution

MoneyUP was developed as a student-focused financial literacy platform designed to simplify personal finance management through budgeting tools, savings tracking, debt management, educational resources, and predictive financial insights. The application helps students create healthier financial habits by providing personalized tools and actionable insights in one centralized platform.

🔧 Technology Stack

Layer	Technology
Frontend	Flutter (Dart)
Backend	FastAPI (Python)
Database	PostgreSQL (Supabase)
Authentication	Supabase Auth
Financial Integration	Plaid API
Server	Uvicorn
UI/UX Design	Figma
Version Control	Git/GitHub

📱 Application Screenshots



🔍 Key Features

- Developed an interactive budgeting system for tracking income, expenses, and spending categories.
- Implemented savings goal tracking to help users monitor financial progress.
- Integrated Plaid API for secure bank account connectivity and transaction synchronization.
- Created financial education modules focused on budgeting, saving, and debt management concepts.
- Built predictive financial insights using machine learning models to estimate future account trends.

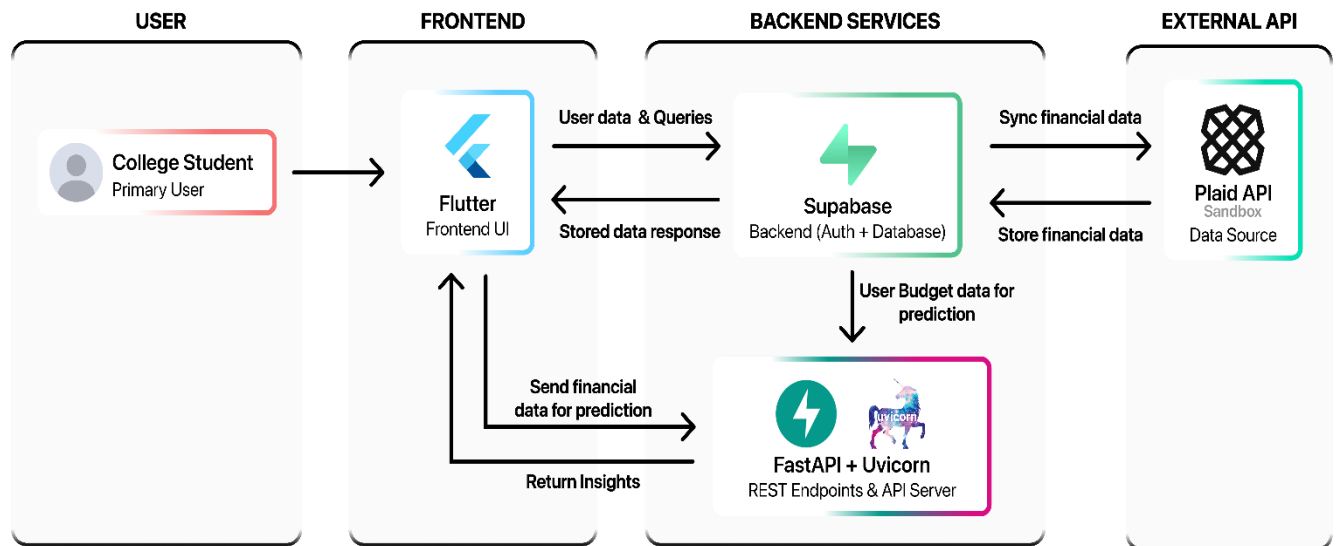
👤 My Role

Project Manager & Full Stack Developer

- Led project planning, task delegation, sprint organization, and team collaboration.
- Designed and developed the Budget Tracker module using Flutter and Dart.
- Built responsive user interfaces based on Figma prototypes.
- Integrated frontend components with backend REST API services.
- Implemented Supabase authentication and PostgreSQL database functionality.
- Assisted with Plaid API integration for financial data synchronization.
- Debugged frontend, backend, and database issues throughout development.
- Documented system architecture and presented technical solutions during project demonstrations.

System Architecture

MoneyUP follows a full-stack mobile application architecture where the Flutter frontend communicates with backend services through REST APIs. User authentication and application data are managed through Supabase, while financial data synchronization is handled through Plaid integration. Machine learning services provide predictive financial insights through a FastAPI backend.



Technical Challenges

Plaid API Integration

Challenge: Synchronizing user financial data between Plaid and the application database.

Solution: Developed backend API workflows to securely exchange Plaid tokens, retrieve transaction data, and store financial information in Supabase.

Debugging Application Issues

Challenge: Connecting Flutter frontend features with backend services for retrieving and processing application data.

Solution: Tested API requests, debugged response handling, and integrated frontend components with backend endpoints to ensure proper communication.

Database Management

Challenge: Maintaining consistent relationships between users, accounts, budgets, and transactions.

Solution: Designed PostgreSQL database tables with relational keys and optimized queries through Supabase.

★ Achievements

- Advanced to semifinals in the 2026 ACC Capital Governor's Cup business competition.
- Presented project at Arkansas Tech University Scholars Symposium.
- Developed as part of Honors Senior Project and Computer Science Capstone.

HONORS SCHOOL MANAGEMENT DATABASE

🔗 Project Links

Source Code

Explore the GitHub repository

<https://github.com/iadams1/School-Management>

📄 Problem

Schools manage large amounts of information related to students, courses, instructors, enrollment, grades, and academic records. Without a properly structured database system, managing relationships between these entities can become inefficient and prone to data inconsistencies.

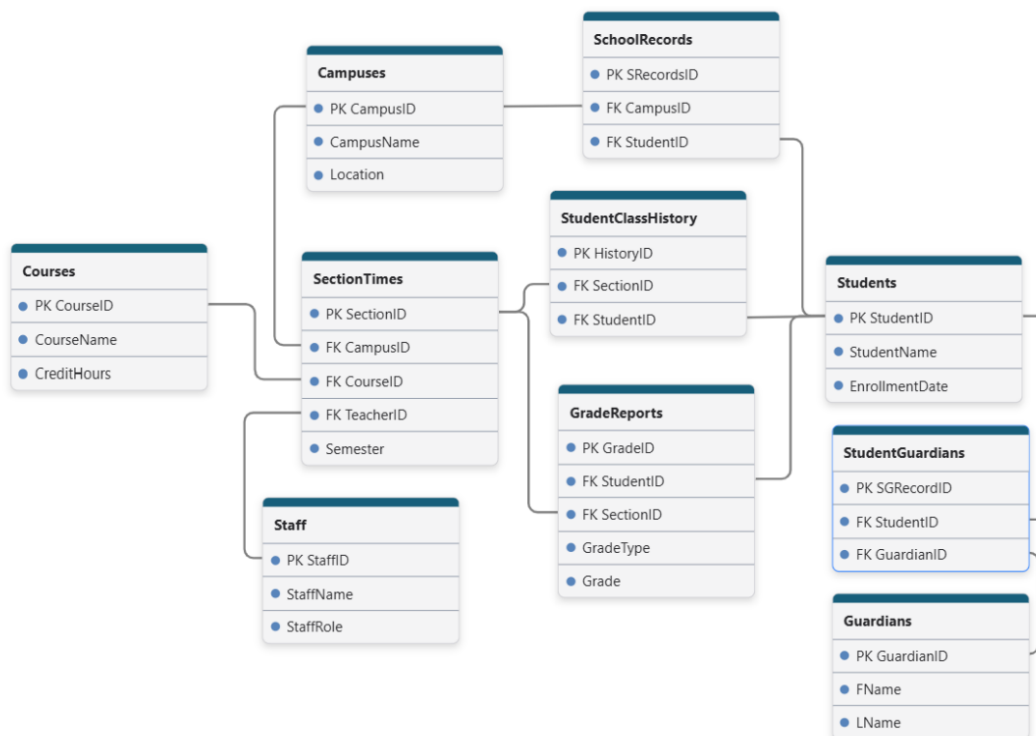
✓ Solution

Developed a relational database system designed to organize and manage academic information through normalized tables, relationships, and SQL queries. The system provides structured storage and efficient retrieval of student, course, instructor, and enrollment data.

🔧 Technology Stack

Layer	Technology
Database	MySQL (SQL)
Programming Language	Python
Security	bcrypt Password Hashing
Database Design	Entity Relationship Diagram (ERD)
Development Tool	MySQL Workbench
Version Control	Git/GitHub

🏗 Database Architecture



🔍 Key Features

- Centralized database system for managing campuses, facilities, budgets, expenses, and events.
- Manage student profiles, enrollment, attendance, payments, academic records, and guardian relationships.
- Supports courses, prerequisites, class scheduling, instructors, enrollment history, and grade reporting.
- Tracks staff roles, qualifications, employment history, campus assignments, and user permissions.
- Implements encrypted data storage, role-based access control, and database audit logging.
- Developed python utilities for generating school reports and monitoring student data.

</> Python Utilities

Password Hashing Utility

Implemented bcrypt-based password hashing to securely store user credentials.

School Report Generator

Automated the generation of administrative reports using database records.

Student Monitoring Tool

Analyzed student information and generated insights from stored academic data.

{ } SQL Queries Screenshots

```
create view StudentsPerformanceSummary as
select
    g.StudentID,
    s.StudentName,
    round(avg(g.Grade), 2) as AverageGrade,
    count(*) as TotalAssignments
from GradeReports g
join Students s
on g.StudentID = s.StudentID
group by g.StudentID;
```

```
create procedure StudentFullClassReport (in g_StudentID int)
begin
    select
        g.SectionID,
        g.StudentID,
        s.StudentName,
        round(avg(g.Grade), 2) as AverageGrade,
        count(*) as TotalAssignments
    from GradeReports g
    join Students s
    on g.StudentID = s.StudentID
    where g.StudentID = g_StudentID
    group by g.StudentID;
end;
```

```
create view MonthlyBudgetModule as
select
    c.CampusID,
    c.CampusName,
    c.MonthlyBudget,
    coalesce(sum(e.Amount), 0) as 'Actual Expenses',
    (c.MonthlyBudget - coalesce(sum(e.Amount), 0)) as 'Remaining Budget',
    month(e.ExpenseDate) as BudgetMonth,
    year(e.ExpenseDate) as BudgetYear
from Campuses c
left join CampusExpenses e
on c.CampusID = e.CampusID
and month(e.ExpenseDate) = current_date()
and year(e.ExpenseDate) = current_date()
group by c.CampusID;
```

```
create view TeacherClassList as
select
    st.SectionID,
    cl.CourseName,
    s.StudentID,
    s.StudentName
from SectionTimes st
join Courses cl
on st.CourseID = cl.CourseID
join GradeReports gr
on st.SectionID = gr.SectionID
join StudentClassHistory sch
on st.SectionID = sch.SectionID
join Students s
on sch.StudentID = s.StudentID
join Staff t
on st.TeacherID = t.StaffID;
```

My Role

Database Developer & Python Developer

- Designed a comprehensive school management database architecture using relational database principles.
- Implemented SQL tables, constraints, and relationships to support academic and administrative operations.
- Built Python-based tools to automate reporting, monitor student records, and improve system functionality.
- Applied security practices through password hashing, encrypted data handling, and user activity tracking.
- Optimized database organization through normalization and structured data modeling.

Technical Challenges

Complex Database Relationships

Challenge: Designing a database structure that could efficiently manage relationships between students, guardians, courses, staff, campuses, and academic records while avoiding data redundancy.

Solution: Applied relational database principles by normalizing tables, implementing primary and foreign keys, and creating structured relationships to maintain data integrity.

Protecting Sensitive Information

Challenge: Managing sensitive user information such as passwords and contact details while maintaining secure access to database records.

Solution: Implemented security measures including encrypted data storage, bcrypt password hashing, user authentication, and audit logging to track database activity.

Automating Administrative Tasks

Challenge: Reducing manual work involved in retrieving and analyzing school information from the database.

Solution: Developed Python utilities for generating school reports, monitoring student data, and processing database information for administrative use.

TASK AND TO-DO LIST MANAGER APP

🔗 Project Links

Source Code

Explore the GitHub repository

<https://github.com/iadams1/Task-Management-App>

📄 Problem

Many users struggle with organizing daily responsibilities and keeping track of important tasks. This project explores the development of a lightweight task management solution that allows users to manage their personal workflow from a mobile device.

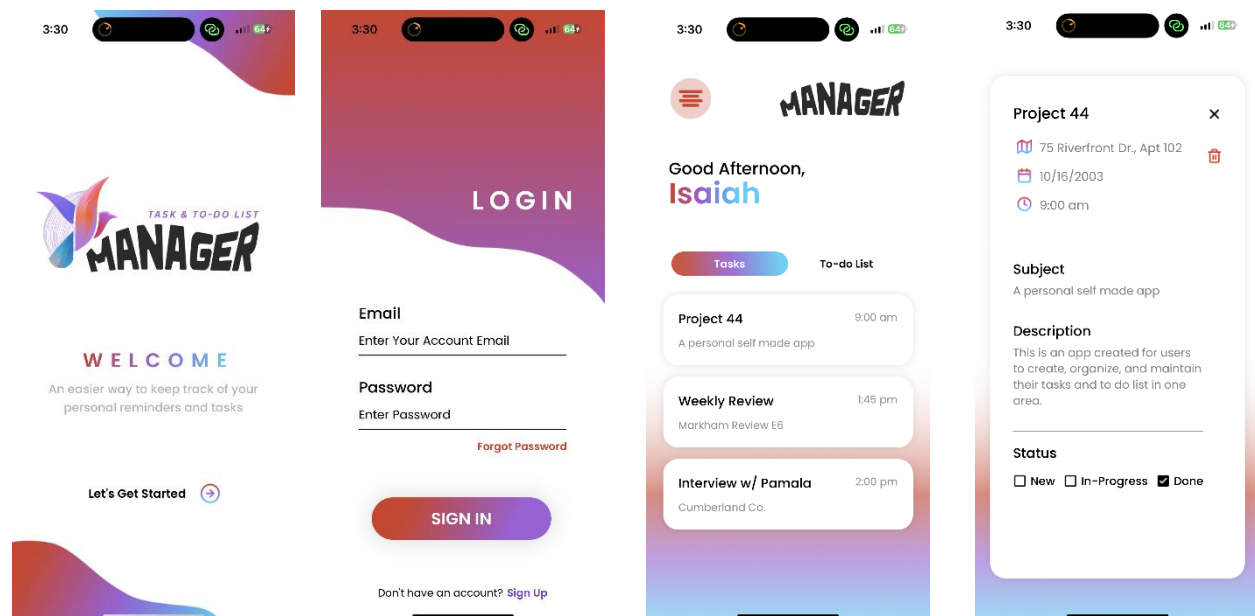
✓ Solution

Developed a mobile task management application with user authentication and task organization features. The application includes a structured interface for navigating between screens and managing task-related information.

🔧 Technology Stack

Layer	Technology
Mobile Framework	React Native
Development Framework	Expo
Programming Language	JavaScript
Backend Services	Appwrite
User Authentication	Appwrite Authentication
UI/UX Design	React Native Components & Canva
Version Control	Git/GitHub

📱 Application Screenshots



Key Features

- User authentication with login and logout functionality
- User onboarding through a welcome screen
- Task creation and deletion functionality
- Task status updates to track completion progress
- Task dashboard for viewing and managing user tasks
- Detailed task view for displaying task information
- Backend integration using Appwrite services
- Mobile-first UI design using React Native and Expo

My Role

Founder & Full Stack Developer

- Designed and developed the mobile application using React Native and Expo.
- Created user authentication workflows including login and logout screens.
- Developed application screens including the welcome page, home dashboard, task list, and detailed task views.
- Implemented task management features including task creation, deletion, and status tracking.
- Integrated Appwrite services for authentication and application data management.
- Tested and refined the application interface to improve usability and user experience.

Technical Challenges

Building a Custom Authentication System

Challenge: Developing user authentication without relying on pre-built authentication services required understanding how user sessions, credentials, and account management work.

Solution: Implemented a custom authentication workflow and structured the application to manage user login states, navigation access, and user-specific information.

Managing Application State and User Interaction

Challenge: Maintaining consistent task information across multiple screens was challenging because users needed to create tasks, view details, and update task statuses.

Solution: Used React state management concepts and component organization to handle user interactions and maintain consistent application behavior.

Current Project Status

This project is currently in development. Completed components include the welcome screen, authentication screens, home dashboard, task list functionality, and detailed task views. Future improvements include completing remaining task management functionality, integrating persistent data storage, and expanding application features.

EVENTLINK – CAMPUS EVENT HUB

🔗 Project Links

Source Code

Explore the GitHub repository

<https://github.com/iadams1/EventLink>

📖 Problem

Students often struggle to discover campus events, organizations, and activities because event information is spread across multiple platforms such as emails, flyers, social media, and university websites. This makes it difficult for students to stay informed, participate in campus activities, and engage with the university community.

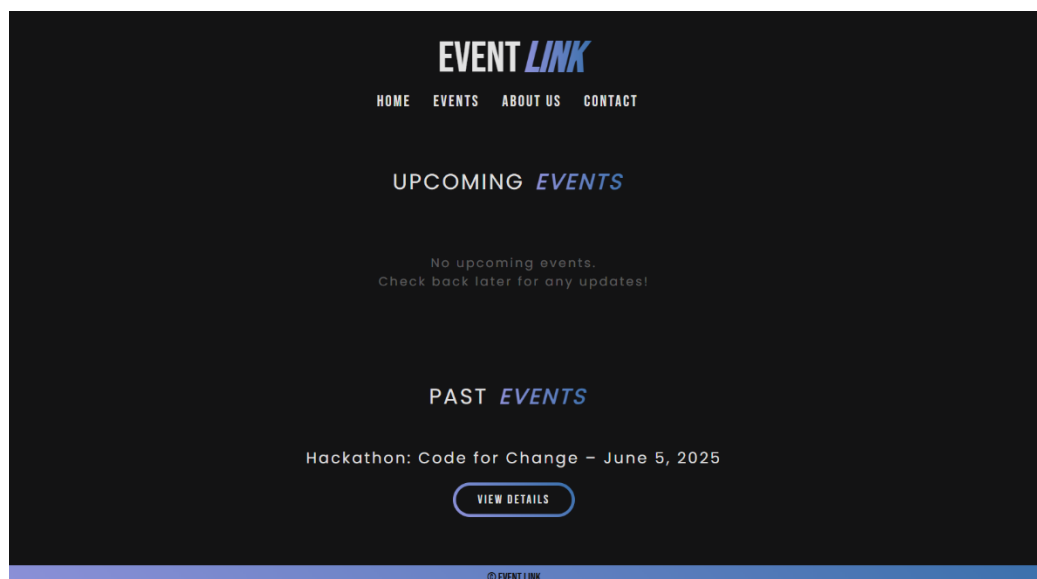
✓ Solution

Developed **EventLink**, a centralized campus event management platform that allows students to discover, view, and interact with campus events in one location. The application organizes event information into an accessible platform, improving event visibility and helping students stay connected with campus activities.

🔧 Technology Stack

Layer	Technology
Frontend	HTML, CSS, JavaScript
Backend Framework	Django (Python)
Programming Language	Python
Database	SQLite
Database Design	Django ORM
Authentication	Django Authentication System
Version Control	Git/GitHub

🖼️ Application Screenshot



Key Features

- Campus event platform for discovering and organizing university activities
- User authentication system for account access and personalized experiences
- Event browsing system allowing users to view available campus events
- Centralized storage of event information including dates, descriptions, and details
- Organized event management structure using Django models and database relationships
- Responsive web interface for improving student accessibility and engagement

My Role

Independent Full Stack Developer

- Designed and developed the EventLink web application using Django and Python
- Built application structure using Django's Model-View-Template (MVT) architecture
- Developed frontend pages using HTML, CSS, and JavaScript
- Implemented database models and relationships using Django ORM
- Created user authentication functionality for account management
- Tested application functionality and debugged issues during development
- Managed project version control using Git/GitHub

Technical Challenges

Learning the Django Framework

Challenge: Django introduced a new development structure through its Model-View-Template (MVT) architecture, requiring an understanding of how models, views, templates, and URLs work together.

Solution: Studied Django's framework structure and applied the MVT pattern to organize application logic, database interactions, and user interface components.

Designing Database Models and Relationships

Challenge: Creating a system to store and display event information required designing database structures that could efficiently organize event data.

Solution: Used Django ORM to create and manage database models while establishing relationships between application components to maintain organized and accessible data.

Connecting Frontend Pages with Backend Functionality

Challenge: Integrating webpage interfaces with backend logic required understanding how user actions, routes, and stored data interact within a web application.

Solution: Implemented Django views, URL routing, and templates to connect user-facing pages with backend functionality and dynamically display application data.
